

CSI247:
Data Structures

Lecture 09 – Sorting Arrays

Department of Computer Science
B. Gopolang (247-275)



Previous Lesson

- Algorithms for searching arrays
  - Linear search
  - Binary search



Today …

Sorting algorithms



Learning Objectives

- By the end of this lecture, students must be:
  - Implement and use these sorting algorithms
    - Bubble sort
    - Insertion sort
    - Selection sort
    - Merge sort

1. Introduction

- Sorting involves traversing a collection
  - A process of arranging/ordering items in some order (Ascending or descending) based on some criteria
  - E.g.:
    - Unsorted: { 12, 7, 23, 9 }
    - Results of sorting: { 7, 9, 12, 23 }
  - The ordering requires comparing values
    - i.e: determining which value should come first

- Note:
  - Sorting can be applied to:
    - A group of primitive values e.g. int, float, etc
    - involves easy comparisons
      - E.g.: 15 > 5?
    - A group of objects belonging to the same class
      - E.g. ordering some LayBuy objects
        - Handled differently in Java (Later)
          - Need for a cleaver way to compare



2. Sorting Algorithms

- There are various sorting algorithms
  - They all arrange items in some order
- Some are better than others in certain situations
- They have different runtimes
  - Some are slow
  - Some quick
- Note:
  - The more efficient the sorting algorithm is, the more complex it will be

a) Bubble Sort

- The simplest sorting algorithm
  - Easy to implement

- **How does it work?**

1. Pass through the entire list (in several passes)
2. Are adjacent items out of sync (wrong order)?
   a) Yes: Swap them
   b) No, leave them

Exercise

- Let us sort this array using bubble sort

63	29	72	85	18

- Note:
  - Array contents will keep on changing as we iterate through the array
  - We need to demonstrate the changes by showing array contents at the end of each pass



Sample method implementation

```
public void bubbleSort (int arr[ ]) {//  assume: ascending order
    int temp;
    for ( int i = 0; i < arr.length; i++ ){//  goes through all array elements
      for (int j = 0; j < arr.length - 1; j++ ){ // compare neigbours
            if ( arr[ j ] > arr[ j+1] ) {  //  Any need to swap neighbours?
                  temp = arr[ j ];       //  swap neighbouring elements
                  arr[ j ] = arr[ j+1];
                  arr[ j+1] = temp;
            }
        }
    }
}
```

Disadvantage

- Inefficient (slow) algorithm
  - Loops through all items, in several passes (iterations)
  - Time complexity: $O(n^2)$



b) Insertion Sort

- Good analogy: sorting of a deck of cards in your hands
- Considers the list to have 2 portions:
  - Sorted part
  - Unsorted part
- For each iteration through the list:
  - Pick an item from the unsorted part
  - Place it at the correct position in the sorted part
- We repeatedly move items from the unsorted part to sorted part until all items are sorted

- Easy to understand

- **How does it work?**

1. Assume first item is first item in the sorted part, and the rest are in the unsorted part
2. Loop through all items in the unsorted part
   - Take each item and insert it at the right place in the sorted part

Exercise

- Let us sort this array using insertion sort

63	29	85	18	72

- When to use it?
  - When list has few items
    - Few overheads
      - Uses less memory
  - When list is nearly sorted
    - Fewer swaps needed
- Disadvantage
  - Still inefficient
    - Time complexity: $O(n^2)$
    - But a little better than bubble sort

Sample method implementation

```
public void insertionSort(int arr[]) {//  assume: ascending order
        int temp, j;
        for (int i = 1; i < arr.length; ++i) {// start from index 1
            j=i;
            temp = arr[i]; //  current item to sort
            while (j>0 && temp < arr[j-1] ) {
                  arr[j] = arr[j-1] )  //  swap adjacent items
                  j --;
            }
           arr[j] = temp; //  place item at the right place
    }
}
```

c) Selection Sort

- Also simple sorting algorithm

- **How does it work?**

1. Select the first element in the array
2. Compare it with all elements in the list.
3. Selected element smaller (or larger) than others?

   Yes: swap the two elements

   No: leave the two elements
4. Repeat the process with the remaining positions in the list till the entire list is sorted.

Exercise

- Let us sort this array using selection sort
- **Rem**: Array contents will keep on changing as we iterate through the array

63	29	72	85	18

Sample method implementation

```
public void selectionSort (int arr[ ]) {//  assume: ascending order
   int minIndex, temp;
  for (int i = 0; i < arr.length - 1; i++) {
      minIndex = i;  // initial index with min value
      for (int j = i + 1; j < arr.length; j++) {
          if (arr[j] < arr[minIndex])
              minIndex = j; // found an index with new min value
      }
      temp = arr[minIndex];
      arr[minIndex] = arr[i];
      arr[i] = temp;
  } }
```



- Disadvantage
  - Still inefficient
    - Loops through all items, several times
    - Time complexity: still $O(n^2)$

d) Merge Sort

- Uses **divide and conquer** principle
  - A problem solving strategy that entails
    - Breaking down a problem into smaller and manageable sub-problems
    - Solving each sub-problem individually
    - Combining sub-problems solutions to solve main problem
- One of the 2 algorithms that uses divide and conquer
  - Another such algorithm: **Quick sort**
- Merge sort is recursive

- **How does it work?**
  1. Keep dividing the array/collection into halves
     - Until each has 1 element
       - (or 2 if some arrays have an odd number of elements)
     - Each will be in sorted order
  2. Recursively, sort each half separately
  3. Merge the sorted halves into bigger arrays
  4. Until we have the final sorted array!



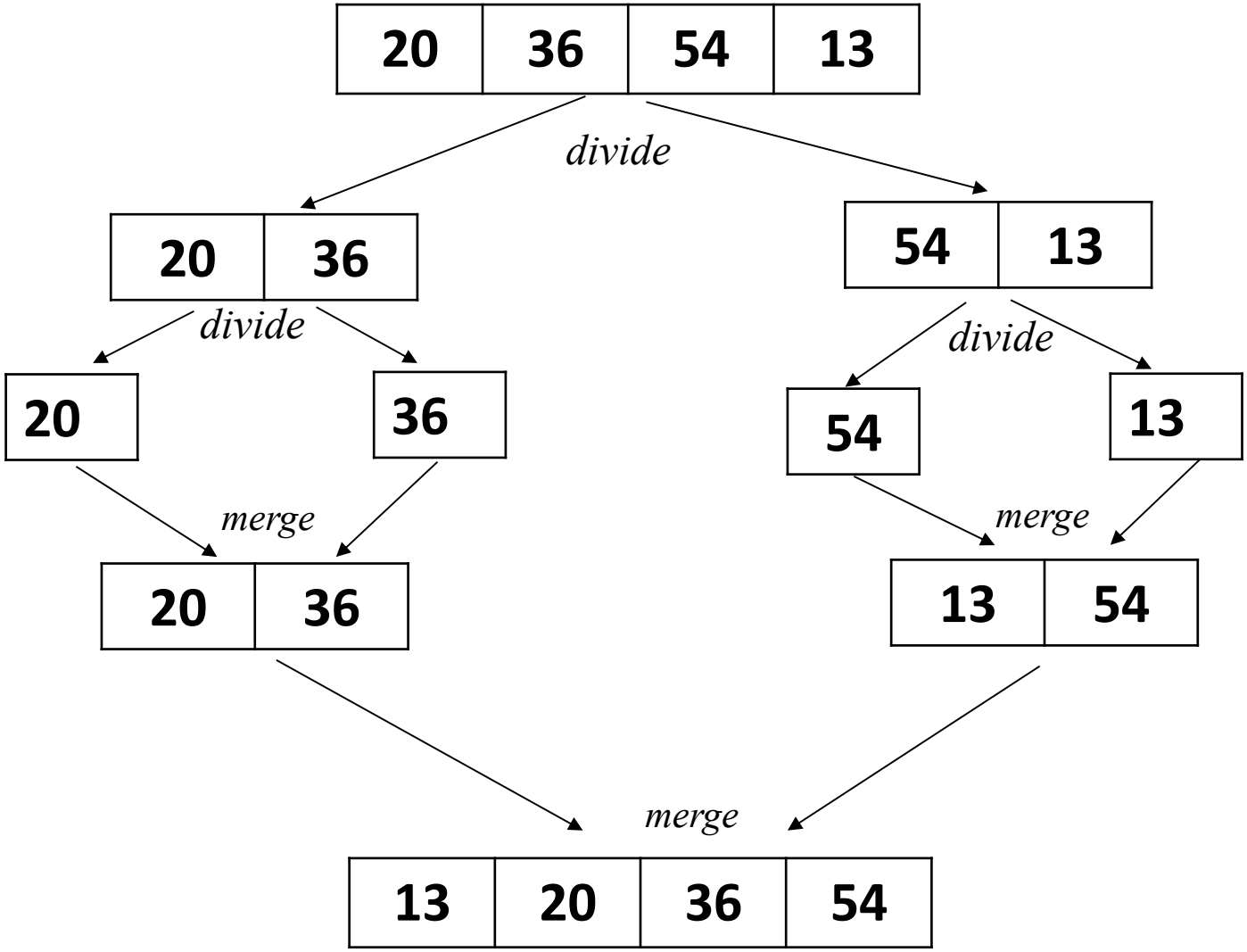
- Merge sort algorithm:
  - If there are items remaining
    - Divide items into half
    - Merge sort the L half of the items
    - Merge sort the R half of the items
    - Merge the L and R halves

Sample method

```
public static void mergeSort(int arr[], int start, int end){
    if (start < end) {
        int mid = (start + end) / 2;  // find the mid point
        mergeSort(arr, start, mid); // Recursively sort L half
        mergeSort(arr, mid + 1, end); // Recursively sort R half
        merge(arr, start, mid, end); // Merge all into 1 array
    }
}
```

Note: merging is implemented separately to ensure a clean & easy to understand code (lab05)

Example


20 36 54 13
divide
20 36
54 13
divide
divide
20
36
54
13
merge
merge
20 36
13 54
merge
13 20 36 54

Exercise

- Sort array below using merge sort

63	29	72	85	18	49	3	54

- 2 operations to merge sort
  - Recursively, **divide** the array into smaller arrays until each has 1 element
    - Each will be sorted
  - **Conquer** the sorting task by merging the small sorted arrays, into 1 sorted array
- More efficient algorithm
  - Keeps on breaking the problem into halves
  - Time complexity: O(n log n)



Summary

- In this lesson, we looked at these sorting algorithms, and their time complexities:
  - Bubble sort: O (n^2)
  - Insertion sort: O (n^2)
  - Selection sort: O (n^2)
  - Merge sort: O (n log n)



Next lesson

- Packages

Q & A

